MODULE 19 · DOMAIN 5 - MODEL SELECTION AND OPTIMISATION

Cost and Token Management

Where the money goes, token budgets, usage tracking, a worked cost model, prompt caching done properly, and caching the prompt versus caching the answer.

DOMAIN 5 of 8 EXAM WEIGHT 16.8% MODULE 19 of 36

admin@sthithapragna.com

You cannot cut a bill you never read. **Meter every feature, then pull the levers that fit.**

Where the money goes

- **Output costs 5x input** on every current model. Long answers are expensive answers.
- **Thinking is billed as output,** and it is on by default on the 5.x models.
- **Every turn resends everything:** tools, system prompt, history, documents.
- **Tool definitions are input tokens,** and so are tool results.
- **The 4.7+ tokenizer** makes the same text about 30% more tokens; images cost more too.

THE TAKEAWAY

Cost per call = input x input price + output x output price, adjusted for cache reads and writes.

Token budgets

- **Per request:** max_tokens caps output, thinking included. Effort sets how much the model tends to use.
- **Per unit of work:** a target cost per conversation, ticket or document, from the requirements.
- **Per context:** trim, compact and prune, so input does not grow without limit.
- **Pre-flight:** count_tokens estimates a request's input before you send it. Free, with its own rate limit.
- **Hard stops:** workspace spend limits; maxBudgetUsd for Agent SDK runs.

> THE TIEBREAKER
>
> A budget nobody checks is a wish. Each budget needs a measure and an action when it is exceeded.

Tracking usage

SOURCE	WHAT IT GIVES YOU
usage on every response	Input, output, cache write and cache read tokens, per call
count_tokens endpoint	An estimate of input tokens before sending
Usage and Cost Admin API	Usage by minute, hour or day; daily cost; Admin key
Console usage export	Usage by API key and model, as a CSV
Workspaces	Separate keys, limits and reports per feature or team

ON THE EXAM

Log usage per call with feature and prompt version. Reconcile with the Usage and Cost API daily.

Cost per call, from the usage fields

```
1  PRICES = {"claude-sonnet-5-5": {"in": 2.00, "out": 10.00, "read": 0.10, "write_5m": 2.50}}
2
3  def cost_usd(model, u):
4      p = PRICES[model]                                    # USD per million tokens
5      return (u.input_tokens * p["in"]
6              + u.cache_creation_input_tokens * p["write_5m"]
7              + u.cache_read_input_tokens * p["read"]
8              + u.output_tokens * p["out"]) / 1_000_000          # output includes thinking
9
10 log.info("claude_call", feature="tenant_assistant", prompt_version=7,
11          request_id=resp._request_id, cost_usd=cost_usd(MODEL, resp.usage))
```

5-8 Four kinds of token, four prices. input_tokens is only the uncached part.

10-11 Tagged by feature and version, so a change shows up the next day.

THE TAKEAWAY

Prices live in configuration and are checked against the pricing page, not hard-coded forever.

A worked cost model: the tenant assistant

PER TURN, SONNET 5.5	WITHOUT CACHING	WITH CACHING
8,000-token tools and system prefix	USD 0.0160	USD 0.0008 (cache read)
3,000 tokens of history, uncached	USD 0.0060	USD 0.0060
500 output tokens, thinking included	USD 0.0050	USD 0.0050
Per conversation (6 turns)	USD 0.162	USD 0.071
Per week (60,000 conversations)	about USD 9,700	about USD 4,250

THE TAKEAWAY

Caching the shared prefix more than halves the bill. The next lever is output length, at 5x the input price.

Prompt caching: the essentials

- **A cached prefix:** tools, then system, then messages, up to a cache_control breakpoint.
- **Up to 4 breakpoints,** or automatic caching with one top-level cache_control.
- **Lifetime:** 5 minutes, refreshed free on each hit; or 1 hour. Writes cost 1.25x or 2x input.
- **Reads:** 0.05x input on Opus and Sonnet 5.5, 0.025x on Fable 5.1, 0.1x on others.
- **Minimum 512 tokens** on current models. Shorter prefixes are silently not cached.

> ON THE EXAM
>
> Cache reads also do not count toward input-token rate limits on current models. Two savings at once.

What silently breaks the cache

CHANGE	WHAT IS INVALIDATED
Tool definitions edited	Everything: tools, system and messages
Anything dynamic early in the prompt	Everything after it, every request
Web search or citations toggled	System and messages
Effort, thinking settings or tool_choice changed	Messages
Images added or removed	Messages

THE TIEBREAKER

Static first, dynamic last. A timestamp at the top of the system prompt defeats caching on every call.

Static first, dynamic last

```
client.messages.create(
    model=TENANT_MODEL, max_tokens=2048,
    tools=TENANT_TOOLS,                              # 1. static: rarely changes
    system=[
        {"type": "text", "text": POLICY_AND_RULES,   # 2. static: same for all
         "cache_control": {"type": "ephemeral"}},
        {"type": "text", "text": pinned(tenancy)},   # 3. dynamic: after it
    ],
    messages=history,                                # 4. grows each turn
)
```

5-6 Breakpoint after the shared prefix: one cache serves every tenant.

7 The per-tenant block from module 8 comes after it, so it never breaks the cache.

THE TAKEAWAY

Check cache_read_input_tokens in production. If it is zero, something before the breakpoint is changing.

Caching the prompt, or caching the answer?

PROMPT CACHING

Anthropic caches your input

- Reuses processing of a repeated prefix
- Claude still generates a fresh answer
- Cheaper, faster input; output still billed
- Works for different questions

RESPONSE CACHING

Your app caches the output

- Stores answers to identical requests
- No Claude call at all on a hit
- Needs a key, expiry and invalidation
- Only for questions that truly repeat

ON THE EXAM

Prompt caching never returns a stored answer. If the same answer to the same question is fine, cache it yourself.

The cost levers, in order

LEVER	WHEN IT FITS
Prompt caching	A large prefix repeats across calls
Message Batches API	Nobody waits; half price, and stacks with caching
The right model	Evals show a cheaper tier passes
Effort and output length	Answers are longer or deeper than needed
Leaner context	History or tool output has grown
Routing	Traffic mixes easy and hard work at volume

THE TIEBREAKER

Measure first, then pull the lever that matches where the tokens actually go.

> A team adds cache_control to its long system prompt, which begins with the line "Current time:" followed by a timestamp. In production, cache_read_input_tokens is always zero.
>
> The system prompt is otherwise identical on every request.

What is the best fix?

A Move the timestamp after the cached prefix, keeping static content first

B Switch to the one-hour cache lifetime

C Add more cache breakpoints to the system prompt

D Send the requests through the Message Batches API instead

◆ PAUSE AND DECIDE

What is the best fix?

A Move the timestamp after the cached prefix, keeping static content first

The prefix stops changing, so it caches.

B Switch to the one-hour cache lifetime

A changing prefix never matches, however long it lives.

C Add more cache breakpoints to the system prompt

Every breakpoint sits after the timestamp.

D Send the requests through the Message Batches API instead

A different lever; the cache still misses.

THE SIGNAL

Cache reads always zero, and the prompt starts with something that changes: static first, dynamic last.

> A customer assistant sends an identical 20,000-token policy prefix with every request. Answers are often three times longer than users need, and the default effort is high. Costs are well over budget.
>
> Quality must stay within the agreed criteria.

Which two changes should the team make? Choose two.

A Raise max_tokens, so that answers complete in a single response

B Cache the shared policy prefix

C Move the assistant to the most capable model

D Ask for concise answers, and evaluate a lower effort level against the criteria

E Repeat the policy in each user message as well, for emphasis

◆ PAUSE AND DECIDE

Which two changes should the team make? Choose two.

A Raise max_tokens, so that answers complete in a single response

Allows even longer output.

B Cache the shared policy prefix

The repeated input drops to a fraction.

C Move the assistant to the most capable model

Higher prices per token.

D Ask for concise answers, and evaluate a lower effort level against the criteria

Cuts output and thinking, the dearer tokens.

E Repeat the policy in each user message as well, for emphasis

Doubles the input.

THE SIGNAL

Two cost sources, two levers: a repeated prefix means cache; long output and high effort mean trim.

> Finance wants the daily Claude cost of each of five features. Today the team sees only the monthly invoice, and all features share one API key.
>
> The team wants accurate, timely numbers.

What should the team do?

A Estimate each feature's cost from the characters in its prompts

B Wait for each monthly invoice, and split it evenly across the features

C Log usage per response with a feature tag; reconcile daily via the API

D Call count_tokens after every response to measure its cost

◆ PAUSE AND DECIDE

What should the team do?

A Estimate each feature's cost from the characters in its prompts

Ignores output, thinking and caching.

B Wait for each monthly invoice, and split it evenly across the features

Late, and wrong by design.

C Log usage per response with a feature tag; reconcile daily via the API

Actual tokens, per feature, every day.

D Call count_tokens after every response to measure its cost

Estimates input only, and adds calls.

THE SIGNAL

Actual cost comes from the usage fields, tagged at source. Separate workspaces or keys per feature help too.

> A team adds cache_control to a 300-token system prompt on Sonnet 5.5. Requests succeed, but cache_creation_input_tokens and cache_read_input_tokens are always zero.
>
> No error is returned.

What is the most likely explanation?

A The cache_control syntax is wrong, which the API ignores silently

B The five-minute lifetime is too short for this traffic

C Prompt caching is not available on this model

D The prefix is under the minimum cacheable length

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The cache_control syntax is wrong, which the API ignores silently

Invalid requests return an error.

B The five-minute lifetime is too short for this traffic

Then writes would still appear.

C Prompt caching is not available on this model

It is, from 512 tokens.

D The prefix is under the minimum cacheable length

Under 512 tokens: no cache, no error.

CURRENCY CHECK

Zero writes and zero reads, no error: check the prefix against the minimum length first.

> An internal help bot receives the same few dozen questions thousands of times a week, and the answers rarely change. The team enabled prompt caching and expected repeated questions to return stored answers at no output cost.
>
> Output costs did not fall.

What should the team understand and do?

A Switch to the one-hour lifetime, so that answers stay cached longer

B Prompt caching reuses input, not answers; add an app-level answer cache

C Send the repeated questions through the Message Batches API

D Add more breakpoints, so that the answers are included in the cache

◆ PAUSE AND DECIDE

What should the team understand and do?

A Switch to the one-hour lifetime, so that answers stay cached longer

Answers are never in the prompt cache.

B Prompt caching reuses input, not answers; add an app-level answer cache

The right tool for repeated answers.

C Send the repeated questions through the Message Batches API

Half price, but still generates each time.

D Add more breakpoints, so that the answers are included in the cache

Breakpoints cover input only.

THE SIGNAL

Same question, same acceptable answer, at volume: a response cache in your app. Prompt caching cannot do it.

> A team runs an experiment that randomly sets each request's effort to low, medium or high. The same long conversation prefix is cached, but the message cache hit rate has collapsed since the experiment began.
>
> Nothing else changed.

What explains it, and what should they do?

A Caching is unreliable during experiments, so turn it off until the test ends

B Move to the one-hour lifetime, so entries survive between requests

C Effort changes invalidate the message cache; fix effort per group

D Shorten the system prompt, so less of it needs to be cached

◆ PAUSE AND DECIDE

What explains it, and what should they do?

A Caching is unreliable during experiments, so turn it off until the test ends

It is reliable; the inputs keep changing.

B Move to the one-hour lifetime, so entries survive between requests

Lifetime is not the problem.

C Effort changes invalidate the message cache; fix effort per group

Stable settings, stable cache.

D Shorten the system prompt, so less of it needs to be cached

Unrelated to the drop.

CURRENCY CHECK

Effort and thinking settings are rendered into the prompt. Change them and the message cache misses.

Module 19 in five lines

- **Output costs 5x input,** thinking is output, and every turn resends everything.
- **Budget per request, per unit and per context;** each with a measure and an action.
- **Track from the usage fields,** tagged by feature and version; reconcile daily.
- **Cache static first, dynamic last;** 512-token minimum; effort changes break it.
- **Prompt caching reuses input,** never answers. Repeated answers need your own cache.

COMING UP

Next, Part 5: tool use from the first call to production.